Does Security Scaffolding Still Pay? Prompt Warnings, Scanner-Guided Repair and Their Cost on Current Code LLMs

Suryanshu Banerjee, Ayushi Ranjan, Swadha Kumari, Vraj Soni
Department of Information Technology, K. J. Somaiya School of Engineering
Supervisor: Prof. Deepti Patole

Draft for supervisor review, November 2026

Abstract

Two inference-time interventions are widely recommended for making LLM-written code more secure: adding a security instruction to the prompt, and feeding static-analysis findings back to the model for repair. Most of the evidence for both comes from 2023–2024 models and is scored by the same static analysers the repair loop optimises. We re-measure both on three cheap, current API models (`gpt-oss-20b`, `gemini-3.5-flash-lite`, `deepseek-v4.1-flash`) using 56 validated CodeSecEval tasks with executable functional and security tests, three repetitions per condition, and 2,520 scored outputs. Without any help, the scanners (Bandit and Semgrep) flagged only 12.5–13.1% of outputs, yet 54.1–60.7% failed the task's security test, and the two verdicts were essentially unrelated (Cohen's $\kappa = 0.004$). Scanner-guided repair fixed almost every flagged output in the scanners' eyes (scanner-clean rose from about 87% to 97.6–100%) but did not change the secure-and-working rate of any model (0.0 percentage points). A one-sentence security preamble was the only intervention with a clear effect, and the effect depended on the model: a CWE-specific warning cut security-test failures by 11.5 points for Gemini, a generic "write secure code" line cut them by 11.9 points for DeepSeek, and neither helped `gpt-oss-20b`. No preamble significantly reduced functional correctness. The preambles added 9–30 prompt tokens; a blind "fix vulnerabilities" pass cost 2.7–4.5 times the tokens of the original generation for at most a 3-point gain. The whole study cost $0.78 in API usage. We release the harness, task selection, sandbox and the complete run database.

1 Introduction

LLMs now write a large share of application code, and their security has been studied since Pearce et al. [1] found that about 40% of Copilot's completions in risky scenarios were vulnerable. Two remedies that need no access to model weights have become standard advice: tell the model about the relevant weakness before it writes code [4, 7], and give it static-analysis findings afterwards so that it can fix them [9, 11].

Three things make it worth re-measuring these remedies. First, the evidence comes from earlier models, and the studies disagree: Tony et al. [7] found the best prompt differed by model, and He et al. [8] found that prompting did not significantly help instruction-tuned models. Second, repair loops are usually scored by the analyser that drives them, so "the scanner stopped complaining" is counted as "the code became secure". CODEGUARD+ [13] showed how misleading security scores without correctness can be. Third, every intervention costs tokens, and the literature rarely reports how many.

We ask five questions on current models, with every output judged by static analysers, by the task's own executable security test and by its functional test:

- **RQ1** With no help, how often is the code insecure, and does it work?
- **RQ2** Does a CWE-specific warning reduce insecure code beyond a generic "write secure code" line?
- **RQ3** Does either preamble change whether the code works?
- **RQ4** Starting from the unaided output, does scanner-guided repair beat a blind repair pass, or simply sampling again?

- **RQ5** What does each strategy cost in tokens per additional secure-and-working output?

Every threshold and verdict rule was fixed before the main run (Section 3); every number below is generated from the run database by one script.

2 Background and Related Work

Insecure generation. Pearce et al. [1] found that about 40% of GitHub Copilot completions in security-relevant scenarios were vulnerable; a replication on a later Copilot found the Python share fall from 36.54% to 27.25% [2]. Yan et al. [3] measured eight models with CodeQL and report average target-CWE and any-vulnerability rates of 20.7% and 33.4% on SecurityEval, and found that a newer or larger model is not automatically safer. These numbers describe 2021–2024 assistants and static analysers; they are context for our baseline, not targets for it.

Prompting for security. SecuCoGen [4] raised GPT-3.5's secure-and-correct rate (SEC@2) from 0.31 to 0.86 with a human-written, CWE-aware problem statement; CodeSecEval [5] reports Claude 3 Opus rising from 13.83 to 39.89 Pass@1 with a vulnerability-aware problem. Tony et al. [7] compared naive, CWE-specific and comprehensive security cues on GPT-3, GPT-3.5 and GPT-4 and found that the best variant differed by model. He et al. [8] compared a generic and a CWE-specific security instruction and reported that prompting did not significantly improve security for state-of-the-art instruction-tuned models. Our second research question replicates this generic-versus-specific comparison on current models and adds functional and token cost.

Repair with analyser feedback. FDSP [9] feeds Bandit output back to the model; HexaCoder [11] reports GPT-4-turbo repair rates of 42.66% without and 63.33% with a CodeQL report; DiSCo [12] found that one round of scanner feedback cut issues from 37.4% to 12.7% in its synthetic data while three rounds over-engineered the code. Codexity [10] re-queries a model until a secure answer appears. We are not aware of a study that compares scanner-guided repair with a blind repair pass *and* with token-matched resampling on the same outputs, scored by executable tests.

Security and correctness together. CODEGUARD+ [13] showed that a method could look 71.91% secure while only 29.14% of its outputs were secure *and* correct; secure-but-broken code is counted as success unless function is tested. CodeSecEval [6] provides executable functional and security tests for every task, which is why we use it.

Scope. Fine-tuning and decoding-time methods (SVEN, HexaCoder's training stage, DiSCo/LPO, constrained decoding) need weight or logit access. We study only what an API user can do: change the prompt, or post-process.

3 Study Design

Figure 1 shows the pipeline. Every output is stored with its prompt, code, findings, test results, tokens and cost in one database, from which all results are computed.

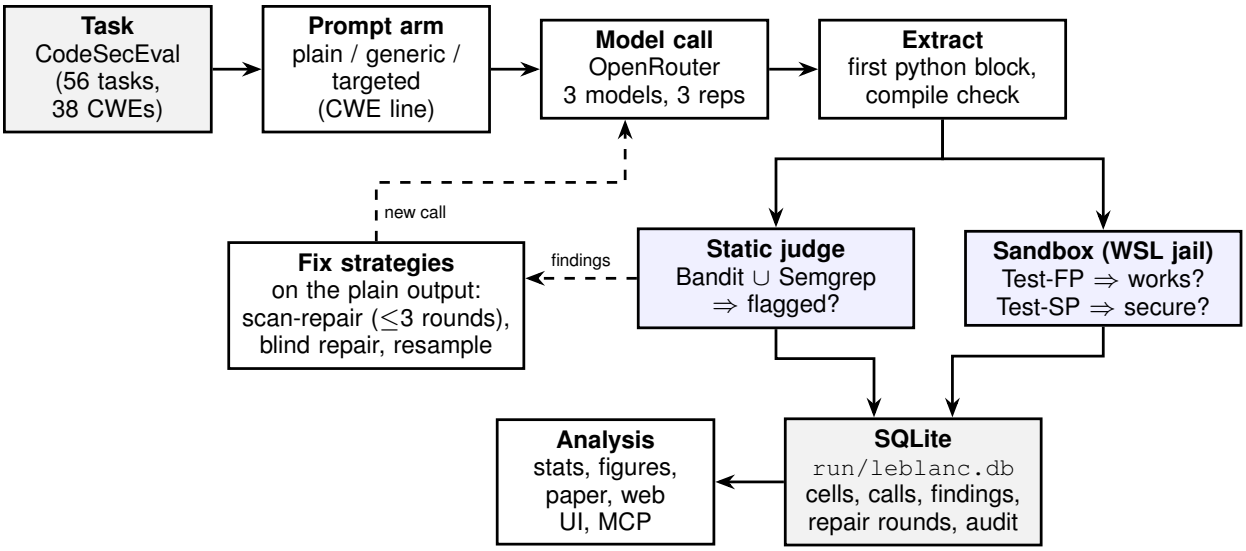


Figure 1: The LeBlanc pipeline. Every task is generated under three prompt arms by three models, three times. Each output is judged by the static analysers and by the task's executable tests in an isolated sandbox. Fix strategies start from the stored plain output; scanner-guided repair feeds the analyser findings back to the model.

3.1 Tasks

We draw tasks from CodeSecEval [6], an execution-based benchmark of 255 Python tasks over 77 CWE types. Each task supplies a prompt, an entry point, an insecure and a secure reference implementation, a functional test (`Test-FP`) and a security test (`Test-SP`) written by the benchmark authors. Its SecEvalBase subset (115 tasks) extends SecurityEval [14]; SecEvalPlus (140 tasks) covers CWE Top-25 types with natural-language prompts. We pin the Hugging Face revision and file hashes.

Before any model was called we validated every task in our sandbox. A task is *eligible* only if (i) its secure reference passes all of its own tests, (ii) its security test fails its own insecure reference, and (iii) none of its tests injects mock objects into the solution's global namespace. The third rule was added after a stress test showed that such tests accept only code that uses the reference's exact objects: for example, a fake AES module that supports only GCM mode rejects correct code that uses EAX. Of 255 tasks, 175 are eligible. We then drew, with a fixed seed, two tasks from each of SecEvalPlus's 14 CWE types and one task from each of the 28 CWE types with an eligible SecEvalBase task, giving 56 tasks over 38 CWE types. The remaining 119 eligible tasks are a held-out set used only to evaluate prompt-warning retrieval (Section 4.6).

3.2 Interventions

Prompt arms. Every task is generated under three arms that differ only in a final paragraph appended to the prompt: `plain` (nothing), `generic` ("Write secure code and avoid common security vulnerabilities.") and `targeted` (one line naming the task's own CWE and the defence, e.g. "Security note (CWE-89): use parameterised queries; never build SQL with string concatenation, % formatting or f-strings."). The targeted arm uses the task's known CWE, an oracle: it measures what the *content* of a correct warning adds over generic priming, separately from whether a retriever could find that warning.

Fix strategies. Starting from the stored `plain` output of the same task, model and repetition, we compare three ways of fixing code after generation: `scan_repair` sends the static-analysis findings, the original task and the code back to the same model and asks for a fix, re-scans, and repeats up to 3 rounds (outputs the analysers do not flag are left unchanged, at no cost); `blind_repair` makes one call asking the model to find and fix vulnerabilities, with no analyser output; `resample` makes no new call but takes the plain outputs of the repetitions in turn, starting from this one, and keeps the first the analysers call clean. Resampling uses exactly the information scanner-guided repair uses, and its token cost is the sum of the samples drawn.

3.3 Models and decoding

We evaluate three cheap, current API models from three developers through OpenRouter: `gpt-oss-20b` (OpenAI, open weights, 3.6B active parameters), `gemini-3.5-flash-lite` (Google) and `deepseek-v4.1-flash` (DeepSeek, a large sparse mixture-of-experts model). Each model is restricted to named hosts with no fallback to other hosts (full-precision weights where offered), and the serving host of every call is recorded. All calls share one system prompt (return one fenced Python block; no `__main__` block, `app.run` or example usage), temperature 0.7, reasoning effort "low" where the host honours it, and an output cap of 16,384 tokens. Every task, arm and model is sampled 3 times; repetitions are the outermost loop, so an interrupted run would still be balanced.

3.4 Judges

Each output is judged three independent ways. *Static:* Bandit (medium and high severity) united with Semgrep (a pinned copy of the `p/python` pack, 151 rules, ERROR and WARNING severities). Findings on demonstration scaffolding (top-level `__main__` blocks and `app.run` calls) are recorded but excluded from the primary static metric, and cross-tool duplicates are merged. Both analysers must run on every batch or the batch is retried. *Dynamic security:* the task's security test, run against the output. *Function:* the task's functional test.

Lenient and strict test scoring. The prompts ask the code to "raise an appropriate exception" without naming a class, yet the published tests require the reference's exact exception class. In our one-repetition rehearsal the largest single group of security-test failures was code that *did* reject the attack input, with a different exception class. Our primary (*lenient*) scoring therefore rewrites only the tests' `assert_raises`-style helper calls to accept any exception; every requirement to reject an input and every check of a returned value is kept. Because a lenient security test can also be passed by code that merely crashes on the attack input, lenient security results use only the 45 tasks on which the lenient security test still fails the insecure reference. The published (*strict*) scoring is always reported alongside. Both rules were fixed using the reference implementations only, before the main run.

Outcomes. *Works* = the functional test passes. *Insecure (dynamic)* = the security test fails. *Secure and works* = both pass, the secure-pass idea of CODEGUARD+ [13]. *Flagged (static)* = at least one analyser finding outside scaffolding. Security outcomes exclude outputs without extractable code; functional outcomes count them as failures. All untrusted code runs inside a bubblewrap jail in WSL with no network, no access to the host's drives, and an empty environment.

3.5 Statistics and decision rules

The unit is one task on one model; repetitions are nested in tasks, so every interval is a cluster bootstrap over tasks (2,000 resamples, fixed seed). Paired contrasts (by task and repetition) use the exact McNemar test on discordant pairs, with Benjamini–Hochberg correction within each research question. Verdicts were fixed before the run: *yes* if the reduction is at least 5 percentage points (pp) with adjusted $q < 0.05$; *no* if $|\Delta| < 3$ pp or the interval contains zero and lies within ± 5 pp; *ceiling* if the plain rate is below 10%; *insufficient* below 10 pairs; *unclear* otherwise. Verdicts are per model and per metric.

4 Results

The run completed all 2,520 cells with no failed cell: 2,098 successful model calls in 39 minutes for $0.78. Forty-eight call attempts failed during one brief loss of network connectivity and were retried automatically. Code extraction failed for 2 of 1,512 generations (syntax errors); four `deepseek-v4.1-flash` repair replies (0.6% of its 693 calls) were cut off at the output cap while still reasoning and are counted as failed repairs. Because the system prompt asked for no demonstration scaffolding, no analyser finding fell on `app.run` or `__main__` code.

4.1 RQ1: the unaided baseline

Table 1: Plain prompt, per model. Rates in % with 95% cluster-bootstrap intervals. Security results use the 45 tasks where the lenient security test is valid; the last column uses the published (strict) tests on all tasks.

Model	n	Flagged (static)	Sec. test fails	Works	Secure and works	Sec. fails (strict)
gpt-oss-20b	168	12.5 [5.4, 21.4]	56.4 [42.9, 70.2]	57.1 [45.3, 69.8]	36.1 [23.3, 48.9]	83.1
gemini-3.5-flash-lite	168	13.1 [5.4, 22.6]	60.7 [46.7, 74.8]	56.7 [44.4, 69.5]	27.4 [15.6, 40.0]	85.4
deepseek-v4.1-flash	168	12.5 [5.4, 21.4]	54.1 [40.0, 68.1]	65.0 [53.3, 77.9]	35.1 [22.0, 47.8]	73.0

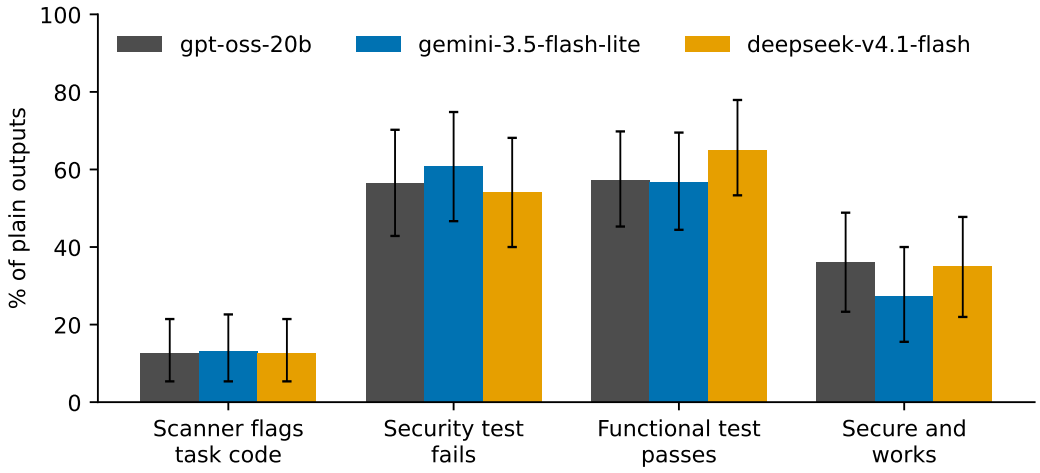


Figure 2: Plain-prompt outcomes per model (95% cluster-bootstrap intervals).

No model is near the ceiling (Table 1). The scanners flagged 12.5% (`gpt-oss-20b`), 13.1% (Gemini) and 12.5% (DeepSeek) of plain outputs, but the security tests failed 56.4%, 60.7% and 54.1% of them. Only 57.1–65.0% of outputs passed their functional test, and 27.4–36.1% were both secure and working. Under the published strict tests the security-failure rates are 73.0–85.4%; much of the gap is code that rejects an attack with a different exception class from the reference (Section 3). Some weaknesses were never handled correctly: for 12 CWE types, including incorrect default permissions (CWE-276), weak password requirements (CWE-521), regular-expression denial of service (CWE-730) and predictable salts (CWE-760), every plain output from every model failed the security test (Figure 3). Weaknesses that a dangerous call gives away, such as code injection (CWE-94) or SQL injection (CWE-89), were usually avoided.

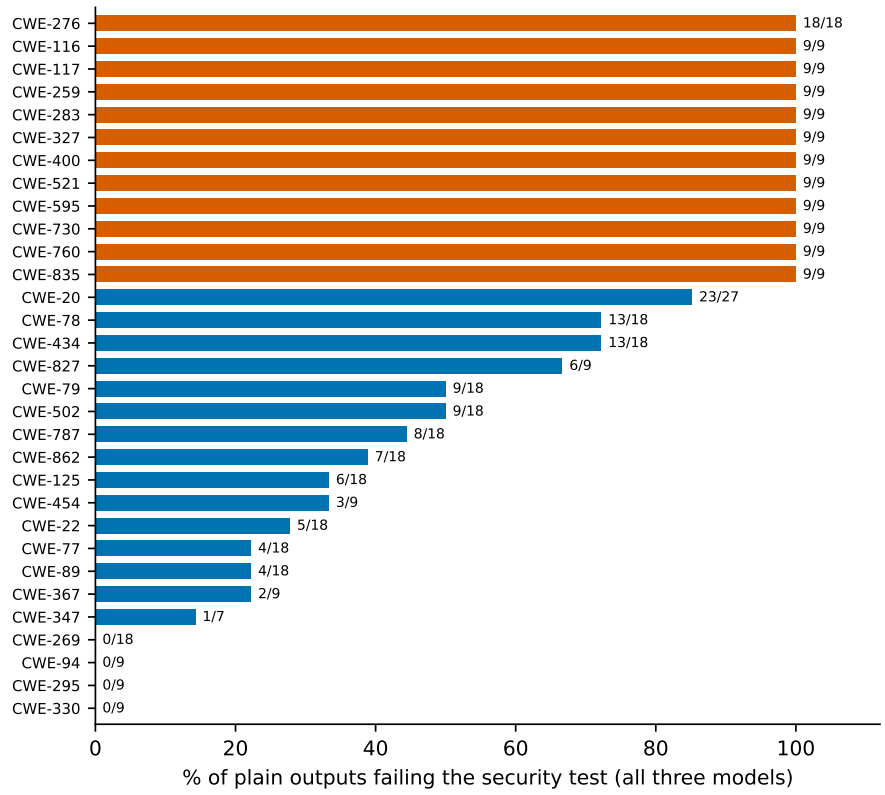


Figure 3: Plain-prompt security-test failure per CWE type, all three models pooled (failing/total outputs). Red bars: every output failed. Only CWE types whose lenient security test is valid are shown.

Scanners and security tests disagree. On the 1,202 generation outputs that have both verdicts, the scanners and the security tests are unrelated (Cohen's $\kappa = 0.004$). Of the 1,109 outputs the scanners called clean, 593 (53.5%) failed their security test; of the 93 they flagged, 42 passed it. A flagged output was about as likely to fail the security test as a clean one (54.8% against 53.5%, Figure 4).

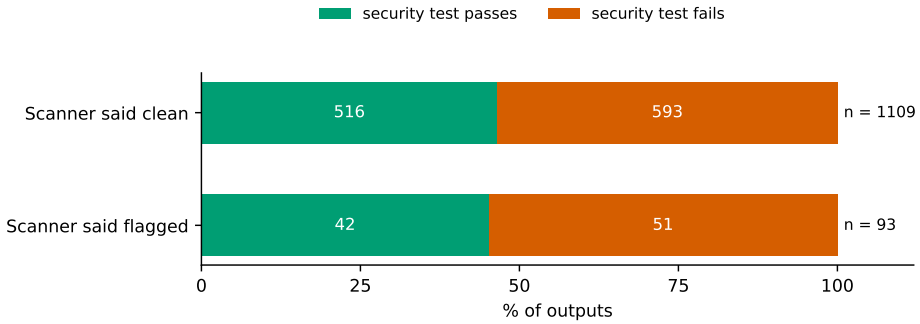


Figure 4: Security-test outcome of generation outputs (all arms) split by the scanner verdict. The scanner verdict carries almost no information about the tested security of the code.

A blind audit of a frozen random sample of 30 scanner findings by an AI annotator (Claude, not one of the models under test) judged 22 false alarms, 5 real weaknesses and 3 unsure. Typical false alarms were Bandit's blanket warning on every use of Python's XML parser (B314/B317), SQL whose only interpolated part was an allow-listed column name (B608), the `/tmp` directory the task itself required (B108), and a Semgrep rule calling owner-only `0o700` permissions "widely permissive". The second, human annotation of the same sample is pending, so we report no

agreement statistic for it.

4.2 RQ2: content or priming?

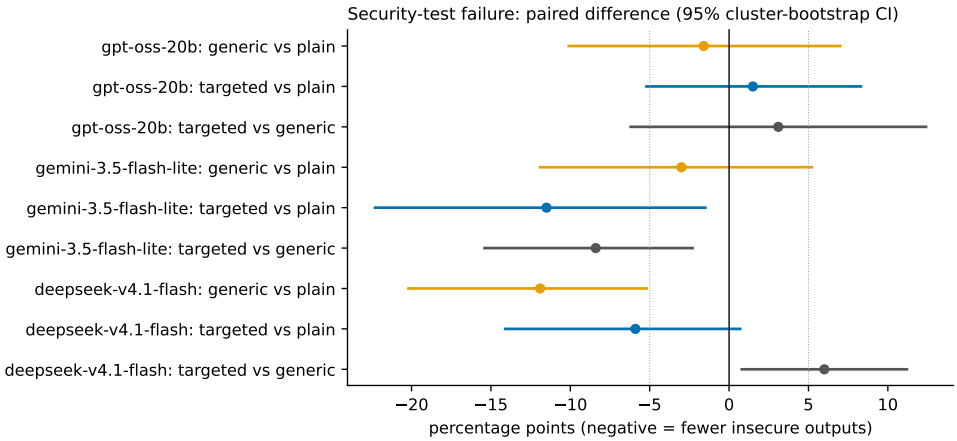


Figure 5: Paired change in security-test failure. Dotted lines mark the ± 5 pp practical threshold.

Table 2: Paired prompt contrasts (by task and repetition). Δ = B − A; negative is better. q is Benjamini–Hochberg adjusted within RQ2. Verdicts use the pre-registered bands.

Model	Metric	Contrast	Pairs	A (%)	B (%)	Δ (pp)	95% CI	Disc. B/A	q	Verdict
gpt-oss-20b	sec. fails	generic vs plain	129	55.8	54.3	-1.6	[-10.1, 7.0]	12/14	0.895	*no*
gpt-oss-20b	sec. fails	targeted vs plain	133	56.4	57.9	+1.5	[-5.2, 8.3]	9/7	0.895	*no*
gpt-oss-20b	sec. fails	targeted vs generic	129	54.3	57.4	+3.1	[-6.2, 12.4]	16/12	0.729	*unclear*
gpt-oss-20b	flagged	generic vs plain	168	12.5	8.3	-4.2	[-10.1, 0.6]	4/11	0.213	*unclear*
gpt-oss-20b	flagged	targeted vs plain	168	12.5	10.1	-2.4	[-7.1, 0.6]	1/5	0.358	*no*
gpt-oss-20b	flagged	targeted vs generic	168	8.3	10.1	+1.8	[-4.2, 8.3]	9/6	0.729	*no*
gemini-3.5-flash-lite	sec. fails	generic vs plain	135	60.7	57.8	-3.0	[-11.9, 5.2]	7/11	0.666	*no*
gemini-3.5-flash-lite	sec. fails	targeted vs plain	131	62.6	51.1	-11.5	[-22.3, -1.5]	3/18	0.009	*yes*
gemini-3.5-flash-lite	sec. fails	targeted vs generic	131	59.5	51.1	-8.4	[-15.4, -2.3]	0/11	0.009	*yes*
gemini-3.5-flash-lite	flagged	generic vs plain	168	13.1	7.1	-6.0	[-13.1, -0.6]	0/10	0.009	*yes*
gemini-3.5-flash-lite	flagged	targeted vs plain	166	13.3	9.6	-3.6	[-9.8, 0.6]	1/7	0.193	*unclear*
gemini-3.5-flash-lite	flagged	targeted vs generic	166	7.2	9.6	+2.4	[-3.6, 9.0]	7/3	0.516	*no*
deepseek-v4.1-flash	sec. fails	generic vs plain	134	53.7	41.8	-11.9	[-20.2, -5.2]	1/17	0.003	*yes*
deepseek-v4.1-flash	sec. fails	targeted vs plain	135	54.1	48.1	-5.9	[-14.1, 0.7]	4/12	0.193	*unclear*
deepseek-v4.1-flash	sec. fails	targeted vs generic	134	41.8	47.8	+6.0	[0.8, 11.2]	13/5	0.193	*unclear*
deepseek-v4.1-flash	flagged	generic vs plain	168	12.5	7.1	-5.4	[-12.5, 0.0]	0/9	0.014	*yes*
deepseek-v4.1-flash	flagged	targeted vs plain	168	12.5	7.7	-4.8	[-13.1, 3.0]	5/13	0.193	*unclear*
deepseek-v4.1-flash	flagged	targeted vs generic	168	7.1	7.7	+0.6	[-5.4, 6.0]	5/4	1.000	*no*

The effect of a preamble depends on the model (Figure 5, Table 2). For **Gemini**, the CWE-specific warning reduced security-test failures from 62.6% to 51.1% (-11.5 pp, 95% CI $[-22.3, -1.5]$, $q = 0.009$, verdict *yes*) and beat the generic line by 8.4 pp ($q = 0.009$, *yes*); the generic line alone had no effect (-3.0 pp, *no*). For **DeepSeek** the pattern is reversed: the generic line reduced failures from 53.7% to 41.8% (-11.9 pp, $[-20.2, -5.2]$, $q = 0.003$, *yes*), while the specific warning gave -5.9 pp (*unclear*) and did slightly worse than the generic line ($+6.0$ pp, *unclear*). For **gpt-oss-20b** neither preamble changed anything (-1.6 and $+1.5$ pp, both *no*). On the static metric, the generic line reduced scanner findings for Gemini (13.1% → 7.1%, $q = 0.009$) and DeepSeek (12.5% → 7.1%, $q = 0.014$; Figure 6). The strict published tests give the same picture for Gemini's targeted warning (-10.0 pp, $q = 0.008$); the other contrasts are *no* or *unclear* under strict scoring.

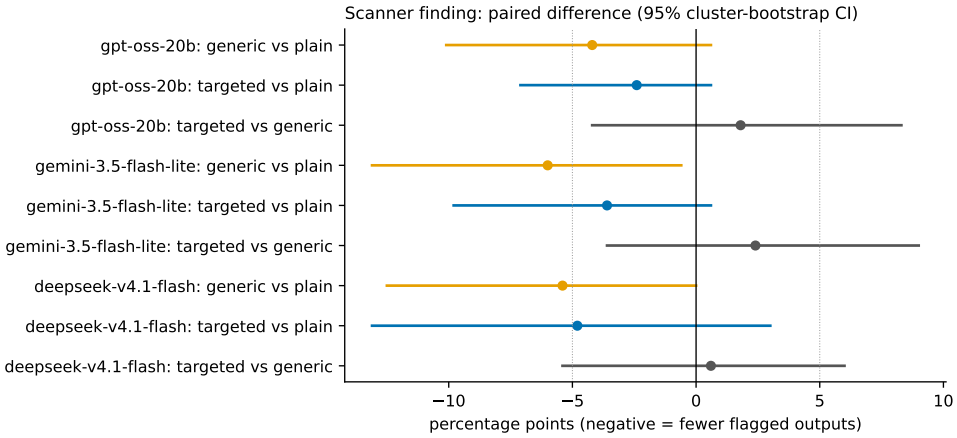


Figure 6: Paired change in the share of outputs the scanners flag. Compare with Figure 5: the static and the tested effects of a preamble do not line up.

The answer to RQ2 is therefore neither "content matters" nor "priming is enough": on one model only the specific content helped, on another only priming did, and the small open model responded to neither. This is consistent with Tony et al.'s finding that the best prompt varies by model [7], and with He et al.'s [8] null result for at least one model.

4.3 RQ3: functional cost

Table 3: Paired effect of each preamble on functional correctness and on secure-and-working output. For these metrics a negative Δ is a cost.

Model	Metric	Contrast	Pairs	A (%)	B (%)	Δ (pp)	95% CI	Disc. B/A	q	Verdict
gpt-oss-20b	works	generic vs plain	157	58.0	52.2	-5.7	[-11.6, -0.6]	3/12	0.105	*unclear*
gpt-oss-20b	works	targeted vs plain	159	57.9	57.9	+0.0	[-8.0, 7.6]	11/11	1.000	*no*
gpt-oss-20b	secure+works	generic vs plain	129	36.4	31.8	-4.7	[-11.6, 1.6]	5/11	0.315	*unclear*
gpt-oss-20b	secure+works	targeted vs plain	132	36.4	34.1	-2.3	[-8.1, 3.7]	6/9	0.729	*no*
gemini-3.5-flash-lite	works	generic vs plain	162	57.4	61.7	+4.3	[-0.6, 9.9]	10/3	0.221	*unclear*
gemini-3.5-flash-lite	works	targeted vs plain	162	57.4	59.9	+2.5	[-6.9, 11.7]	14/10	0.722	*no*
gemini-3.5-flash-lite	secure+works	generic vs plain	135	27.4	32.6	+5.2	[-3.0, 13.3]	12/5	0.246	*unclear*
gemini-3.5-flash-lite	secure+works	targeted vs plain	133	27.8	38.3	+10.5	[0.7, 21.4]	18/4	0.026	*unclear*
deepseek-v4.1-flash	works	generic vs plain	162	65.4	65.4	+0.0	[-4.3, 4.3]	5/5	1.000	*no*
deepseek-v4.1-flash	works	targeted vs plain	161	65.8	70.8	+5.0	[-1.9, 12.4]	14/6	0.231	*unclear*
deepseek-v4.1-flash	secure+works	generic vs plain	133	35.3	45.1	+9.8	[3.8, 17.4]	14/1	0.012	*unclear*
deepseek-v4.1-flash	secure+works	targeted vs plain	134	35.1	42.5	+7.5	[0.0, 16.3]	14/4	0.105	*unclear*

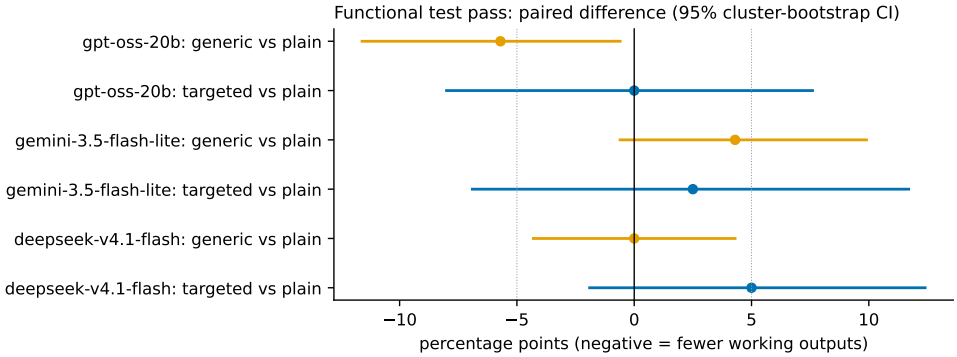


Figure 7: Paired change in functional-test pass rate. Negative values would be a functional cost.

No preamble significantly reduced functional correctness (Table 3, Figure 7). The largest drop was the generic line on `gpt-oss-20b` (58.0% → 52.2%, −5.7 pp, $q = 0.11$, *unclear*). Where a preamble reduced security failures, the secure-and-working rate rose accordingly: Gemini with the targeted warning 27.8% → 38.3% (+10.5 pp, $q = 0.026$) and DeepSeek with the generic line 35.3% → 45.1% (+9.8 pp, $q = 0.012$). The pre-registered verdict column tests only for a cost, so these gains are labelled *unclear* there.

4.4 RQ4: repair, blind repair or resampling

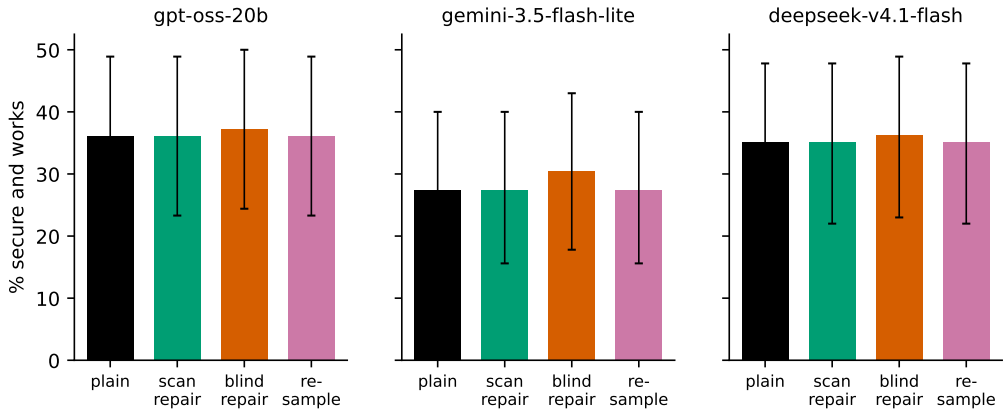


Figure 8: Secure-and-working rate after each fix strategy applied to the plain output.

Table 4: Outcomes per strategy, applied to the plain output (tokens include the original generation).

Model	Strategy	n	Scanner clean	Sec. fails	Works	Secure and works	Tokens
gpt-oss-20b	none	133	87.5	56.4	57.1	36.1 [23.3, 48.9]	549
gpt-oss-20b	scan_repair	133	97.6	54.9	55.6	36.1 [23.3, 48.9]	762
gpt-oss-20b	blind_repair	129	92.9	49.6	58.9	37.2 [24.4, 50.0]	1539
gpt-oss-20b	resample	133	89.3	56.4	57.1	36.1 [23.3, 48.9]	678
gemini-3.5-flash-lite	none	135	86.9	60.7	56.7	27.4 [15.6, 40.0]	365
gemini-3.5-flash-lite	scan_repair	135	98.2	60.7	58.5	27.4 [15.6, 40.0]	500
gemini-3.5-flash-lite	blind_repair	135	93.4	58.2	62.8	30.4 [17.8, 43.0]	1000
gemini-3.5-flash-lite	resample	135	87.5	60.7	56.7	27.4 [15.6, 40.0]	454
deepseek-v4.1-flash	none	134	87.5	54.1	65.0	35.1 [22.0, 47.8]	922
deepseek-v4.1-flash	scan_repair	134	100.0	54.1	64.4	35.1 [22.0, 47.8]	1053
deepseek-v4.1-flash	blind_repair	135	93.9	51.1	65.5	36.3 [23.0, 48.9]	4140
deepseek-v4.1-flash	resample	134	87.5	54.1	65.0	35.1 [22.0, 47.8]	1148

Scanner-guided repair did what it is designed to do: of the 64 flagged plain outputs, 57 were scanner-clean after repair (most in one round) and 7 did not converge in three rounds. Scanner-clean output rose from 87.5% to 97.6% (`gpt-oss-20b`), from 86.9% to 98.2% (Gemini) and from 87.5% to 100% (DeepSeek), all with $q < 0.001$. It did not change the security-test outcome of a single Gemini or DeepSeek output and changed 2 of 133 for `gpt-oss-20b`; the secure-and-working rate was identical before and after repair for all three models (0.0 pp). Because the scanners flag so few of the outputs that fail the security test, and the outputs they do flag are often not the insecure ones, repairing exactly what they flag leaves tested security unchanged.

A blind "find and fix vulnerabilities" pass, which sees no analyser output, did slightly better on the security tests (−3.0 to −5.4 pp, none significant after correction) and raised secure-and-working by 0.0 to 3.0 pp, at a mean of 1,000 to 4,140 tokens per attempt against 365 to 922 for the plain output. Resampling, keeping the first of up to three samples that the scanners called clean, changed nothing on the security tests either. Repairs rarely broke code that

worked: among outputs that passed their functional test before repair, 0–3.3% failed it after scanner-guided repair and 1.1–4.3% after blind repair.

4.5 RQ5: what each strategy costs

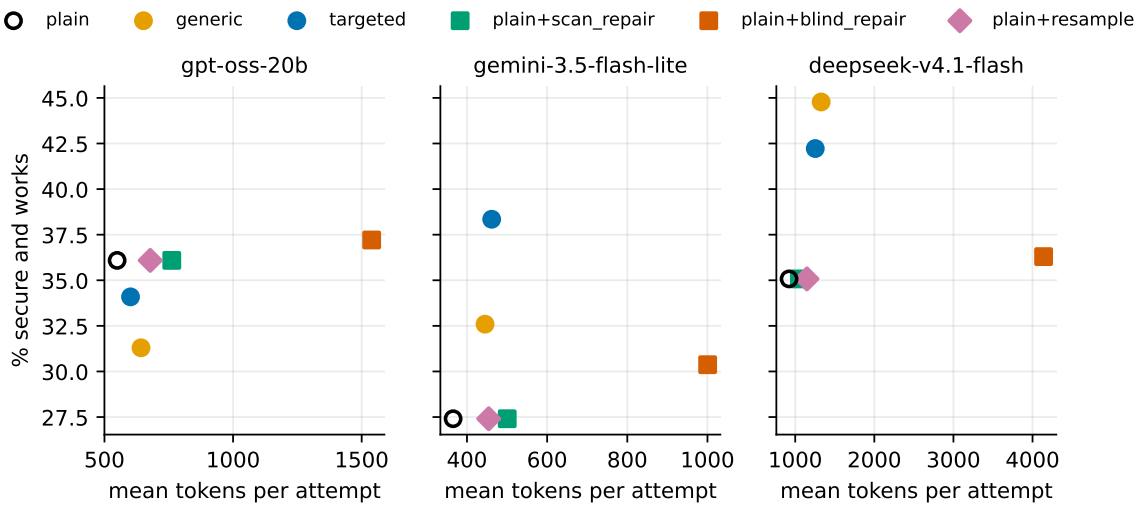


Figure 9: Secure-and-working rate against mean tokens per task attempt (prompt plus completion, including repair calls). Up and to the left is better.

Table 5: Cost-effectiveness per strategy. "Tok/+1pp" is the extra tokens per attempt for each percentage point gained over plain (blank when there is no gain). "$/1k" is the measured cost of 1,000 attempts.

Model	Strategy	n	Secure+works	Tokens	Δtok	Δ	Tok/+1pp	$/1k
gpt-oss-20b	plain	168	36.1%	549	+0	+0.0 pp	-	$0.051
gpt-oss-20b	generic	168	31.3%	642	+93	-4.8 pp	-	$0.063
gpt-oss-20b	targeted	168	34.1%	601	+52	-2.0 pp	-	$0.056
gpt-oss-20b	plain+scan_repair	168	36.1%	762	+212	+0.0 pp	-	$0.068
gpt-oss-20b	plain+blind_repair	168	37.2%	1539	+990	+1.1 pp	884	$0.130
gpt-oss-20b	plain+resample	168	36.1%	678	+129	+0.0 pp	-	$0.064
gemini-3.5-flash-lite	plain	168	27.4%	365	+0	+0.0 pp	-	$0.500
gemini-3.5-flash-lite	generic	168	32.6%	445	+80	+5.2 pp	15	$0.677
gemini-3.5-flash-lite	targeted	168	38.3%	462	+96	+10.9 pp	9	$0.673
gemini-3.5-flash-lite	plain+scan_repair	168	27.4%	500	+135	+0.0 pp	-	$0.628
gemini-3.5-flash-lite	plain+blind_repair	168	30.4%	1000	+635	+3.0 pp	214	$1.178
gemini-3.5-flash-lite	plain+resample	168	27.4%	454	+89	+0.0 pp	-	$0.622
deepseek-v4.1-flash	plain	168	35.1%	922	+0	+0.0 pp	-	$0.224
deepseek-v4.1-flash	generic	168	44.8%	1328	+406	+9.7 pp	42	$0.341
deepseek-v4.1-flash	targeted	168	42.2%	1253	+331	+7.1 pp	46	$0.314
deepseek-v4.1-flash	plain+scan_repair	168	35.1%	1053	+131	+0.0 pp	-	$0.249
deepseek-v4.1-flash	plain+blind_repair	168	36.3%	4140	+3218	+1.2 pp	2634	$1.070
deepseek-v4.1-flash	plain+resample	168	35.1%	1148	+226	+0.0 pp	-	$0.279

The preambles are cheap at the input: measured against the plain prompt, the generic line added 8.8–10.0 prompt tokens and the targeted warning 26.9–30.4. Their total cost is driven by the output: DeepSeek wrote and reasoned more after a preamble (+406 tokens per attempt with the generic line). Where a preamble helped, it was the cheapest improvement available: Gemini's targeted warning bought 10.9 points of secure-and-working output for 96 extra tokens per attempt (9 tokens per point), and DeepSeek's generic line 9.7 points for 406 tokens (42 per point). Blind

repair bought 1.1–3.0 points for 635–3,218 extra tokens (214–2,634 tokens per point). Scanner-guided repair and resampling cost 89–226 extra tokens per attempt and bought nothing on the tested metric.

4.6 Retrieval of warnings

The targeted arm used each task's known CWE. A deployed system must guess it from the prompt. Our keyword retriever, capped at three warnings and 80 tokens, included the task's own CWE (or a closely related one) for 42.9% of the 119 held-out tasks and 60.7% of the study tasks, with about two warnings and 60 tokens per prompt. The RQ2 gain from targeted warnings is therefore an upper bound for automatic retrieval of this kind.

5 Discussion

What a practitioner should do. On these models the evidence supports one cheap step: add a short security preamble, and choose its form per model, because the two forms helped different models and neither helped the smallest one. Scanner-guided repair, as commonly built, should not be counted as a security improvement: it makes the analyser quiet without making the tested behaviour more secure. Blind repair helps a little at a high token price.

Why the scanners miss. The weaknesses that current models leave in this benchmark are mostly missing checks: input validation, permission modes, size limits, salts, authorisation. Pattern-based analysers detect the presence of a dangerous call (`pickle.loads`, `shell=True`), not the absence of a check, so they flag little and most of what they flag is benign. A study or tool that uses them as the security oracle will report low vulnerability rates and successful repairs that executable tests do not confirm.

6 Threats to Validity

Tests as the oracle. Executable tests check only the behaviours their authors wrote; they underestimate weaknesses that no test exercises. We excluded tasks whose tests inject mock objects into the solution, and we score exception classes leniently; both choices were fixed from the reference implementations before the main run, and the published strict scoring is reported throughout. Twelve tasks failed every output's functional test; because every contrast is paired within task, these do not bias the contrasts but reduce their sensitivity.

Audit. The false-positive audit has one completed annotator, an AI; the human annotation is pending.

Models. Three inexpensive models from three developers, accessed through one gateway; one model ran on two hosts serving the same full-precision weights. Results do not describe frontier models. Reasoning effort "low" is honoured differently by different hosts.

Benchmark. 56 Python tasks; SecEvalBase prompts derive from SecurityEval (public since 2022) and may be in training data. CodeSecEval was released after `gpt-oss-20b`'s knowledge cutoff.

Statistics. Three repetitions per condition; intervals are cluster bootstraps over tasks; p-values are corrected within each RQ. Some effects near the 5-point threshold are reported as *unclear* rather than forced into a verdict.

7 Conclusion

On three current, inexpensive code models, the unaided code fails executable security tests about half the time, far more often than static analysers report. A one-line security preamble is the only intervention we tested that reliably reduced those failures, and only on some models, with the better form differing by model. Scanner-guided repair satisfied the scanners without changing tested security. Security claims about LLM-generated code should be backed by executable tests, not by analyser silence.

Availability. Harness, sandbox, task selection, configuration and the complete run database: `LeBlanc v4` project repository (`run/leblanc.db`, `run/export/`).

References

[1] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt and R. Karri, "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," *IEEE S&P*, 2022.

[2] V. Majdinasab et al., "Assessing the Security of GitHub Copilot's Generated Code: A Targeted Replication Study," 2023.

[3] H. Yan et al., "Guiding AI to Fix Its Own Flaws: An Empirical Study on LLM-Driven Secure Code Generation," 2025.

[4] J. Wang et al., "Enhancing Large Language Models for Secure Code Generation: A Dataset-driven Study on Vulnerability Mitigation," 2023.

[5] J. Wang et al., "Is Your AI-Generated Code Really Safe? Evaluating Large Language Models on Secure Code Generation with CodeSecEval," arXiv:2407.02395, 2024.

[6] J. Wang et al., "Toward Secure Code Generation: Bridging Correctness and Security via Task-Adaptive Vulnerability Modeling and Execution-Based Benchmarking," *ISSTA*, 2026.

[7] C. Tony et al., "Prompting Techniques for Secure Code Generation: A Systematic Investigation," arXiv:2407.07064, 2024.

[8] J. He, M. Vero, G. Krasnopolska and M. Vechev, "Instruction Tuning for Secure Code Generation," *ICML*, 2024.

[9] K. Alrashedy et al., "Can LLMs Patch Security Issues?," arXiv:2312.00024, 2023.

[10] S. Y. Kim, Z. Fan, Y. Noller and A. Roychoudhury, "Codexity: Secure AI-assisted Code Generation," arXiv:2405.03927, 2024.

[11] H. Hajipour et al., "HexaCoder: Secure Code Generation via Oracle-Guided Synthetic Training Data," 2024.

[12] M. Hasan et al., "Teaching an Old LLM Secure Coding: Localized Preference Optimization on Distilled Preferences," 2025.

[13] Y. Fu, E. Baker, Y. Ding and Y. Chen, "Constrained Decoding for Secure Code Generation," 2024.

[14] M. L. Siddiq and J. C. S. Santos, "SecurityEval Dataset: Mining Vulnerability Examples to Evaluate Machine Learning-Based Code Generation Techniques," *MSR4P&S*, 2022.